KONGU ENGINEERING COLLEGE

DEPARTMENT OF INFORMATION TECHNOLOGY

SALESFORCE DEVELOPER PROJECT DOCUMENTATION

Revolutionizing Agriculture with AgriEdge Or-Mange Ltd: A Salesforce-Driven Order Management Solution

PROJECT DOCUMENTATION

Prepared by

SAVITHRA

24ITR125

Table of Contents

• Introduction to the Project

• Project Goals

• Phase 1 – Business Requirements and Project Planning

•Phase 2 – Salesforce Backend Implementation and Setup

• Phase 3 – Salesforce Interface and Application Setup

•Phase 4 – Data Handling, Validation and Security

•Phase 5 – Deployment Planning and System Maintenance

•Implementation Screenshots

•Limitations of the Current Implementation

•Proposed Future Improvements

•Summary and Conclusion

1. Introduction to the Project

AgriEdge Order Management System is a Salesforce-based application used to manage customers, orders, order items, inventory, and shipments in one place.

We used Salesforce objects, formulas, validation rules, security features, Apex classes, triggers, and Process Builder to reduce manual work and automate the order process.

2. Project Goals

- Manage customer orders in one system.
- Track order items, inventory, and shipments.
- Automatically calculate prices and totals.
- Automate order-related tasks and updates.
- Control user access using roles and profiles.
- Test the Apex functionality using test classes..

3. Phase 1 – Business Requirements and Project Planning

3.1 Business Requirement Identification

The main business requirement is to manage the complete order process in Salesforce by connecting customer details, orders, order items, inventory, and shipment information. The system also provides different access permissions based on user roles and automates repetitive tasks, such as task creation and order-status updates, to improve efficiency and reduce manual work.

3.2 Scope of the Proposed System

Area	Scope
Customer management	Account used as the customer reference for AgriEdge Orders.
Order management	Order status, date, amount, payment status, shipping address and discounted total.
Order items	Product, quantity, unit price and calculated total price.
Inventory	Product, stock quantity, reorder level, warehouse location and calculated stock status.
Shipment	Order, tracking number, carrier and shipment status.
Security	Roles, profiles and field-level security.
Automation	Process Builder, Apex classes and Apex triggers.
Testing	Apex test class covering implemented Apex functionality.

3.3 Salesforce Data Structure

The data model establishes relationships between the main objects in the AgriEdge system. The AgriEdge Order object is linked to the Account object through Customer__c. The AgriEdge Order Item object is connected to AgriEdge Order through Order__c and to Product through Product__c. Similarly, AgriEdge Inventory is linked to Product using Product__c. These relationships help organize customer, order, product, and inventory information within Salesforce.

Object	Important Relationships / Purpose
AgriEdge Order	Customer__c → Account; parent record for order processing.
AgriEdge Order Item	Order__c → AgriEdge Order; Product__c → Product.
AgriEdge Inventory	Product__c → Product; stores stock and reorder information.
AgriEdge Shipment	Order__c → AgriEdge Order; stores carrier and shipment status.

3.4 User Access and Security Configuration

The project uses a role hierarchy and custom profiles. The planned hierarchy is Kongu Engineering college → CEO, with Sales Representative and Warehouse Manager under CEO, and Finance Team under Sales Representative. Profiles Platform 2 and Platform 3 were configured with object permissions according to the project requirements. Field-level security was also configured for selected AgriEdge Order fields.

4. Phase 2 – Salesforce Backend Implementation and Setup

4.1 Development Environment and Tools

The AgriEdge application was developed and configured in a Salesforce Developer Edition environment. The development process involved using Lightning Experience, Setup, and Developer Console to create custom objects, fields, user profiles, and roles. Process Builder was used to automate selected tasks, while Apex classes and triggers were implemented to handle business logic. Apex test classes were also executed to verify the functionality of the implemented code.

4.2 Object and Field Configuration

Object	Key Fields / Configuration
AgriEdge Order	Order Number (Auto Number); Customer__c; Order_Status__c; Order_Date__c; Total_Amount__c; Payment_Status__c; Shipping_Address__c; Discounted_Total__c.
AgriEdge Order Item	Order Item Number (Auto Number); Order__c; Product__c; Quantity__c; Unit_Price__c; Total_Price__c.
AgriEdge Inventory	Inventory Number (Auto Number); Product__c; Stock_Quantity__c; Reorder_Level__c; Warehouse_Location__c; Stock_Status__c.
AgriEdge Shipment	Shipment Number (Auto Number); Order__c; Tracking_Number__c; Carrier__c; Status__c.

4.3 Formula-Based Calculations

Field	Formula	Purpose
Discounted_Total__c	Total_Amount__c - (Total_Amount__c * 0.1)	Calculates a 10% discounted order total.
Total_Price__c	Quantity__c * Unit_Price__c	Calculates the order-item total.
Stock_Status__c	IF(Stock_Quantity__c <= Reorder_Level__c, "Low", "Sufficient")	Shows whether stock is low or sufficient.

4.4 Workflow Automation Using Process Builder

A Process Builder named 'Order Task Creator' was configured for AgriEdge Order records. The process starts when an AgriEdge Order record is created, checks the Order Status condition for 'New', and invokes the Apex action 'Create Task for New Order' using the OrderTaskCreator Apex class.

4.5 Apex Programming and Trigger Implementation

Component	Purpose
OrderTaskCreator	Invocable Apex action that creates a high-priority task for a new order and assigns it to users with the Platform 1 profile.
OrderStatusUpdater	Changes eligible New orders to Processing.
OrderTotalUpdater	Aggregates AgriEdge Order Item totals and updates the parent order amount and payment status.
OrderItemTrigger	Recalculates the related order total after order-item insert, update, delete or undelete.
OrderEmailSender	Loads order information for email-related processing; current implementation contains the order query and placeholder email logic.
OrderPaymentStatusTriggers	Updates payment status based on whether the order total is greater than zero.
AgriEdgeOrderShipmentHelper	Handles order-status/payment-status-related shipment and order-item processing.
AgriEdgeOrderTriggerHelper	Invokes shipment processing after order update and contains a recursion guard.
AgriEdgeOrderTrigger	After-update trigger on AgriEdge Order.

No asynchronous Apex class is included in the implemented scope documented here.

5. Phase 3 – Salesforce Interface and Application Setup

5.1 AgriEdge Application Configuration

The AgriEdge Lightning app was created to provide navigation to the main business areas, including AgriEdge Orders, AgriEdge Order Items, AgriEdge Inventories, AgriEdge Shipments, Reports and Dashboards.

5.2 Page Layout and Interface Configuration

Document the page layouts and Lightning Pages that are actually configured in the org. Dynamic Forms and LWC should only be listed as implemented if they were created.

Current implementation record: No LWC was developed for the documented scope.

5.3 User and Role Configuration

User / Role	Profile	Purpose
John Production Engineer Sandbox 1	Platform 1	Sales Representative role / primary business user.
Quality Inspector Mike	Platform 2	Warehouse Manager role.
Plant Manager Albert	Platform 3	Finance Team role.

6. Phase 4 – Data Handling, Validation and Security

6.1 Salesforce Record Management

The project demonstration uses Salesforce records created during configuration and testing. A Data Import Wizard or Data Loader migration process was not documented as part of the implemented scope.

6.2 Data Tracking and Quality Controls

These features should be documented only if configured in the org. They are not included as completed components in this project documentation.

6.3 Profile Permissions and Field Access

Platform 2 was configured with Read/Create/Edit access for AgriEdge Inventory, Account and AgriEdge Shipment, and read-only access for AgriEdge Order and AgriEdge Order Items. Platform 3 was configured with Read/Create/Edit access for AgriEdge Order and AgriEdge Order Items.

Selected AgriEdge Order fields were restricted using field-level security according to the project requirements, including Payment Status, Discounted Total and Total Amount for the specified profiles.

6.4 Testing and Code Coverage Results

The AgriEdgeOrderTests test class was executed after resolving trigger-recursion issues. The final documented test run reported 7 tests with 0 failures and overall code coverage of 76%.

Test Item	Expected / Observed Result
Apex test class	AgriEdgeOrderTests
Tests executed	7
Failures	0
Overall code coverage	76%
Status	Passed

6.5 Functional Test Cases and Results

The following evidence should be inserted into the final document. Each screenshot should show the input/action and the resulting output where applicable, as required by the project guidelines.

Test Case	Input / Action	Expected Output
Order creation	Create an AgriEdge Order with required fields.	Order record created; formula fields display correctly.
Discounted total	Enter Total Amount = 7000.	Discounted Total = 6300.
Order item total	Enter Quantity = 2 and Unit Price = 25.	Total Price = 50.
Inventory status	Set Stock Quantity at or below Reorder Level.	Stock Status = Low.
Process Builder task	Create an AgriEdge Order with Order Status = New.	Process invokes OrderTaskCreator and creates a task.
Order-item total update	Insert/update/delete order items.	Parent order total is recalculated.
Order trigger	Update Payment Status to Paid.	Order-status processing occurs according to the implemented trigger/helper logic.
Apex test execution	Run AgriEdgeOrderTests.	7 tests pass; 0 failures; 76% overall coverage.

7. Phase 5 – Deployment Planning and System Maintenance

7.1 Application Deployment Approach

The AgriEdge application is currently configured in a Salesforce Developer Edition environment for development and demonstration purposes. If the application needs to be transferred to another Salesforce organization, the required metadata can be deployed using tools such as Change Sets or other suitable Salesforce deployment methods. This will help move the application configuration to another environment when required.

7.2 System Monitoring and Maintenance Activities

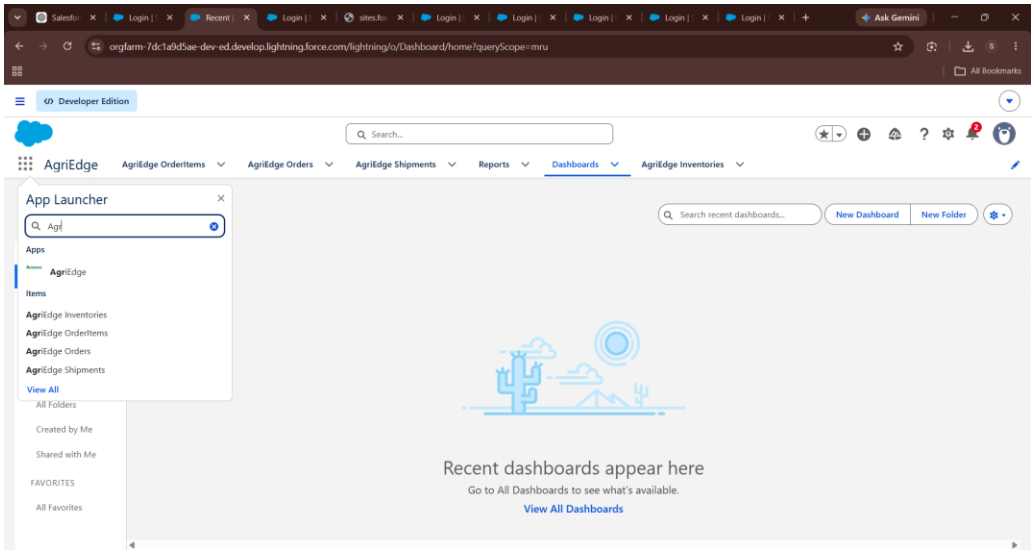
- Review Process Builder/Apex behavior after changes to order fields or status values.
- Monitor Apex test results and maintain the required code coverage.
- Review user profiles and field-level security when responsibilities change.
- Validate formula fields whenever the underlying field names or business rules change.
- Use debug logs and test data to investigate automation failures.

7.3 Issue Identification and Resolutio

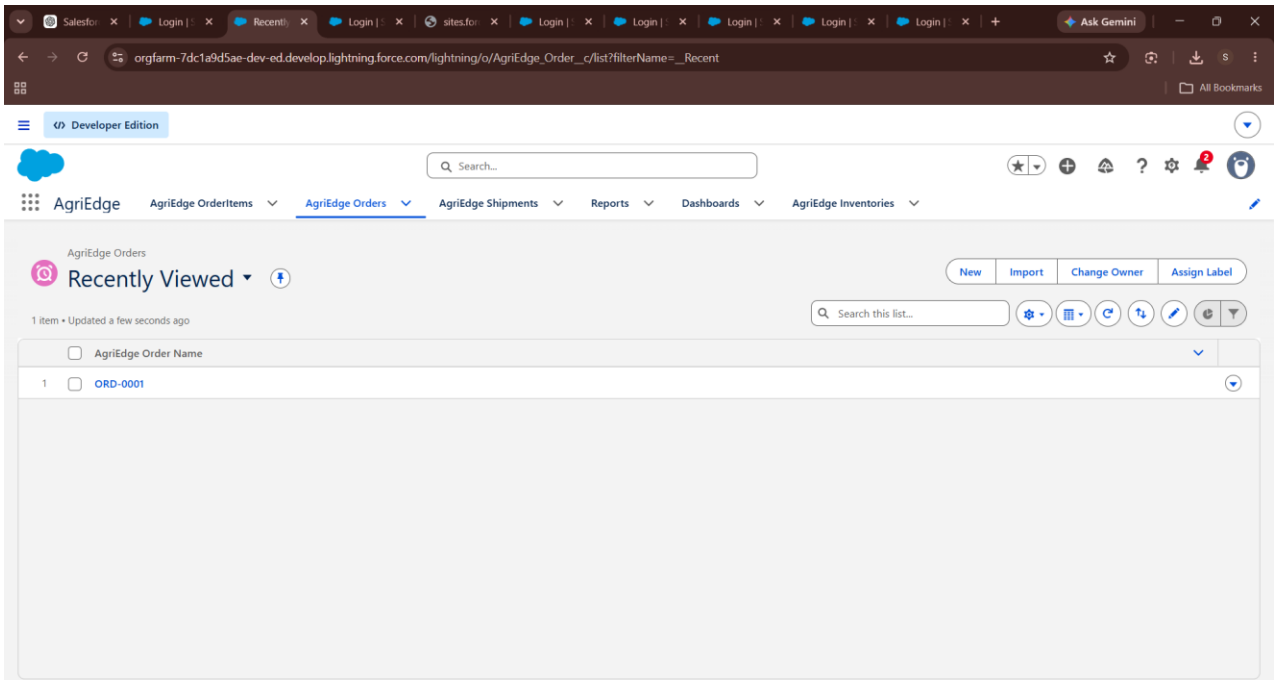
During the development phase, a trigger recursion issue was identified while running the Apex tests. This issue was resolved by implementing a recursion guard in the AgriEdgeOrderTriggerHelper class. After applying the fix, the Apex tests were executed again, and all seven tests passed with zero failures.

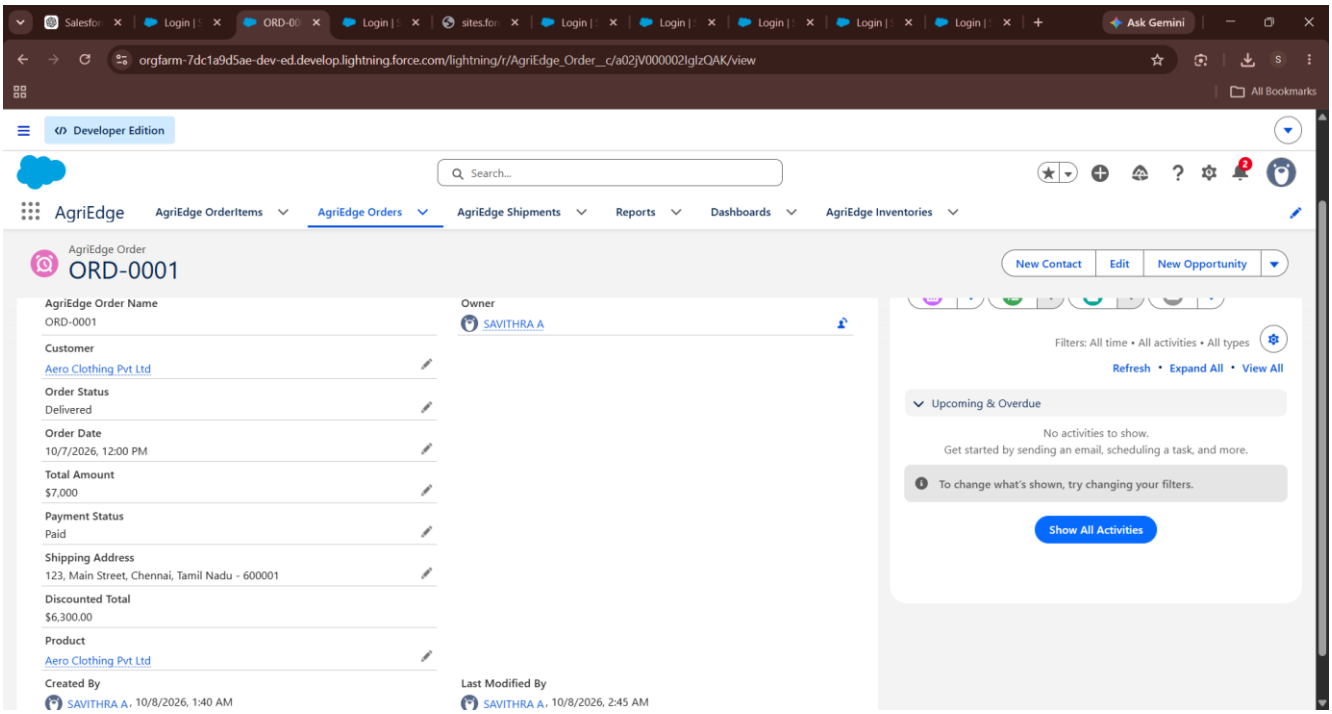
8. Implementation Screenshots

[SCREENSHOT 1 – AgriEdge App Launcher / Navigation]

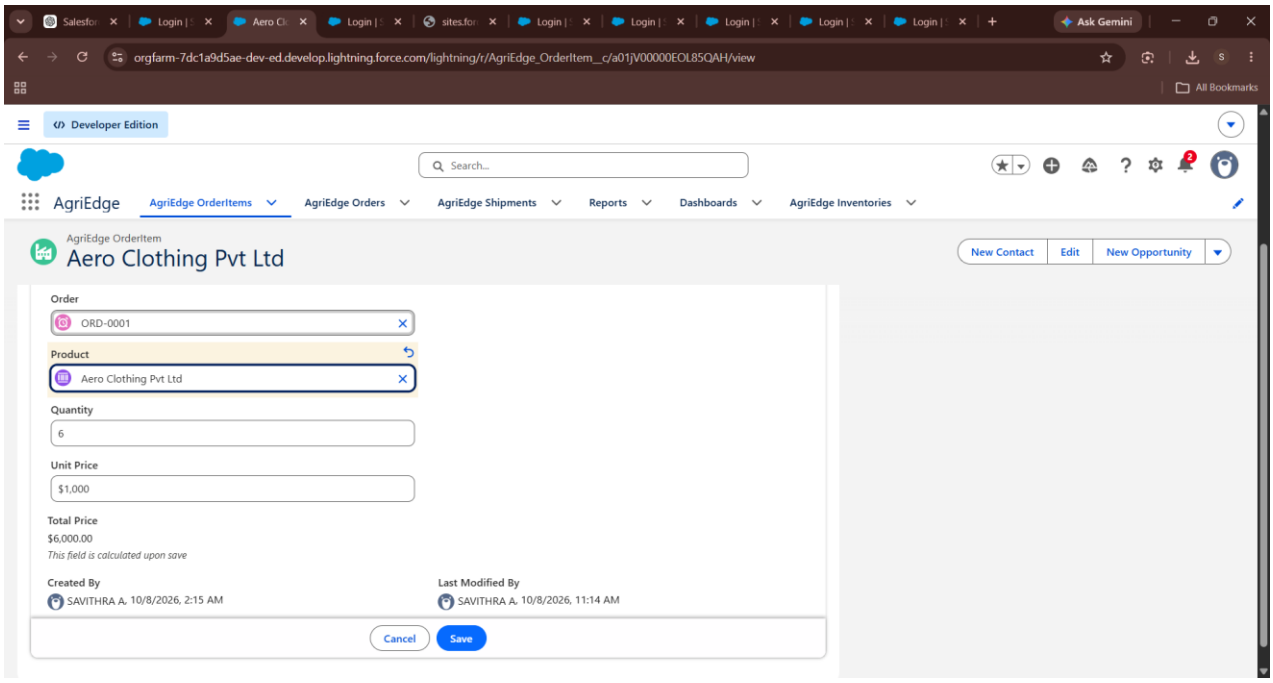


[SCREENSHOT 2 –AgriEdge Order record]

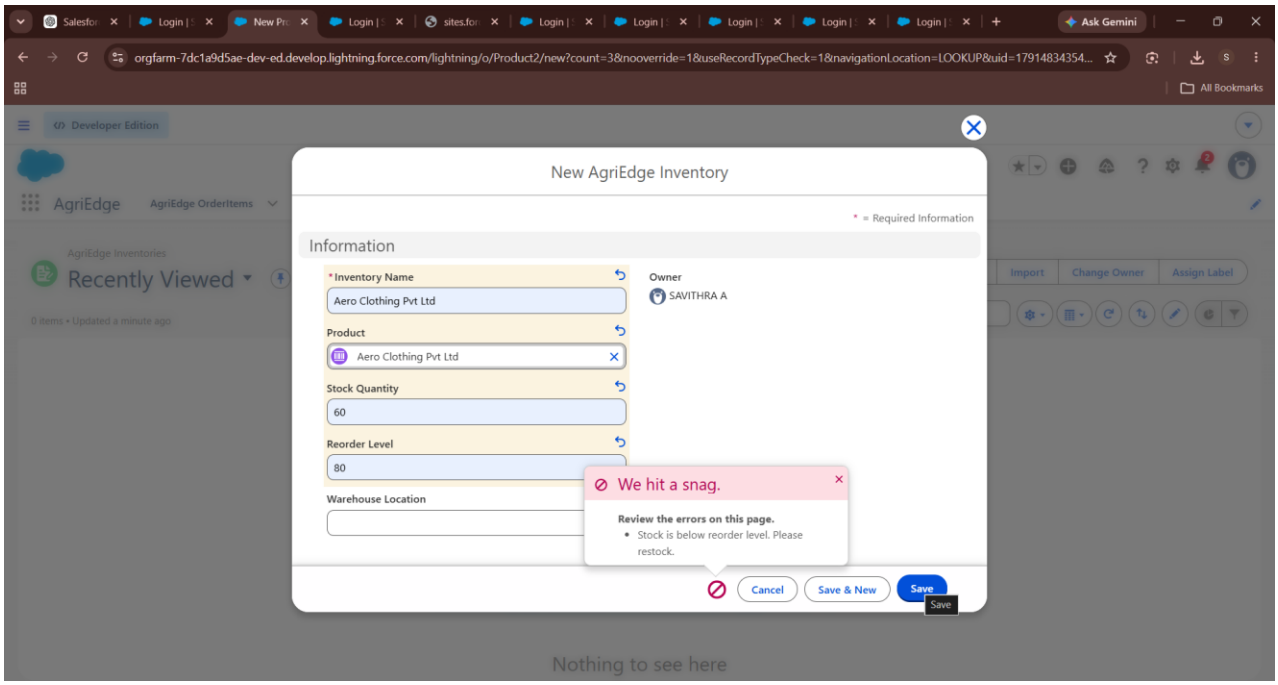




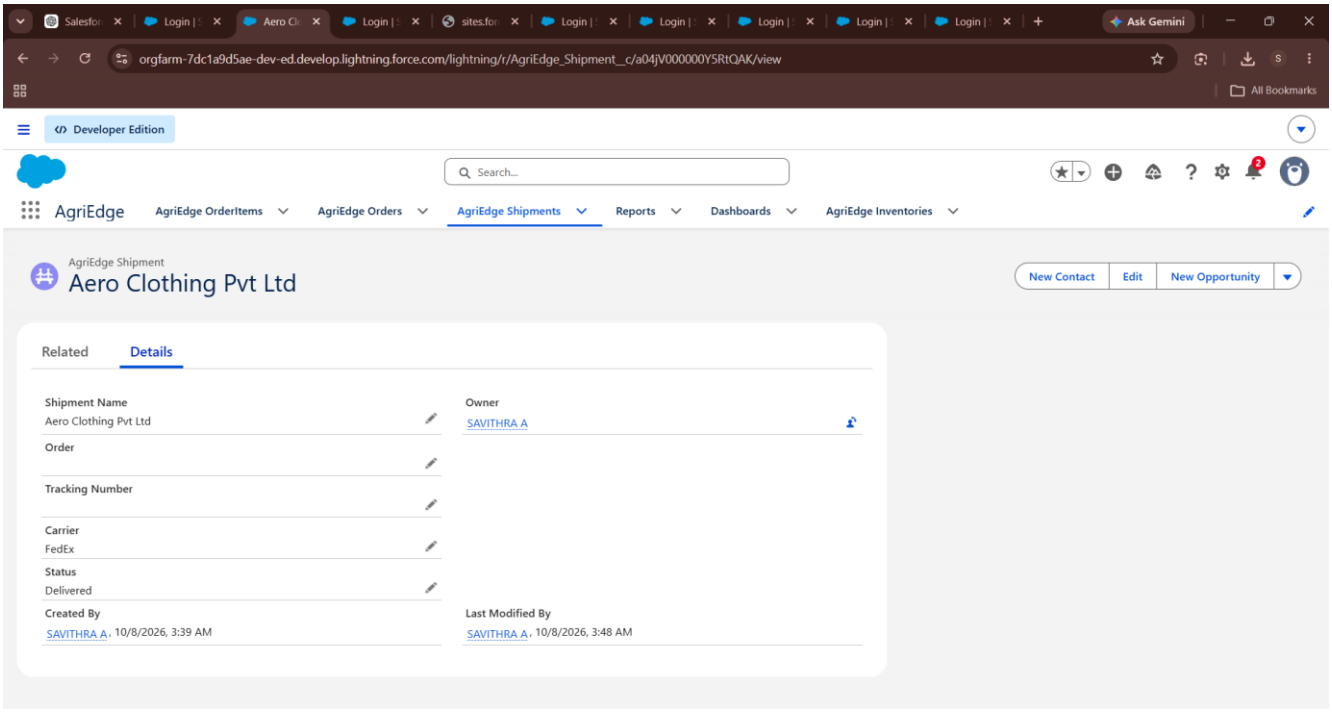
[SCREENSHOT 3 – AgriEdge Order Item Record]



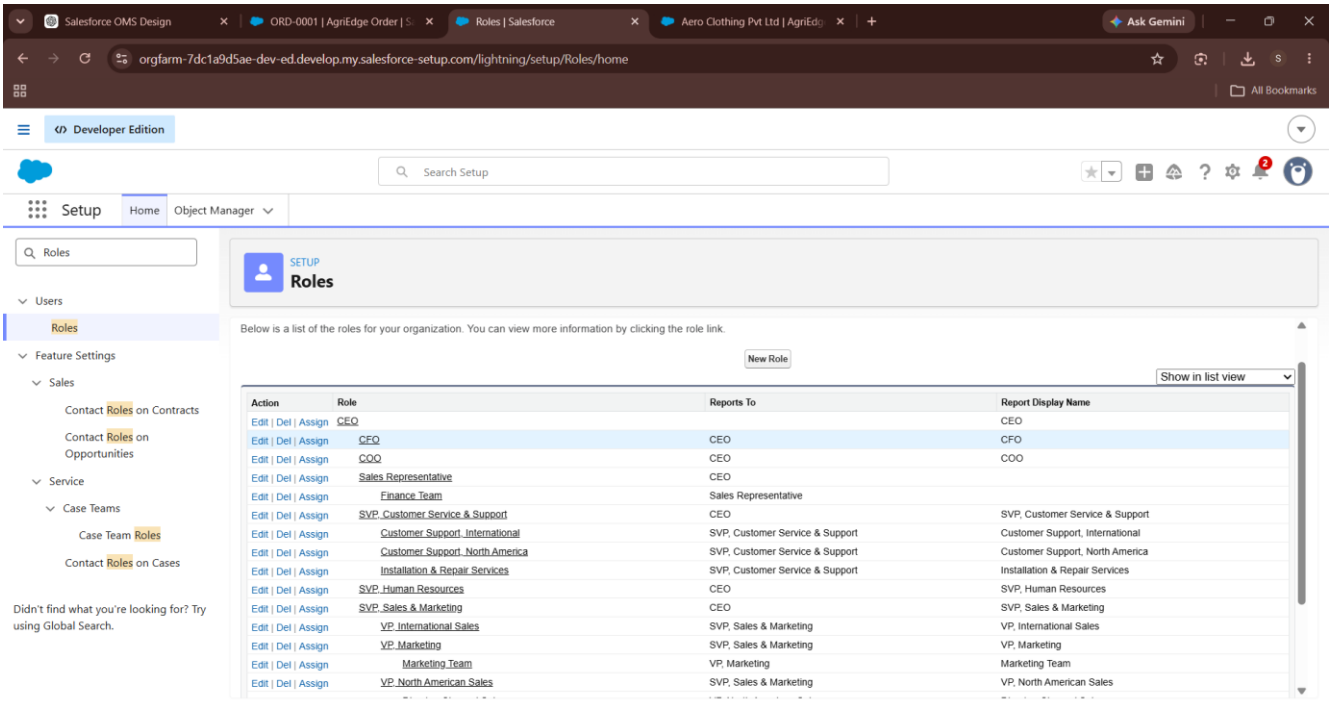
[SCREENSHOT 4 – AgriEdge Inventory Record]



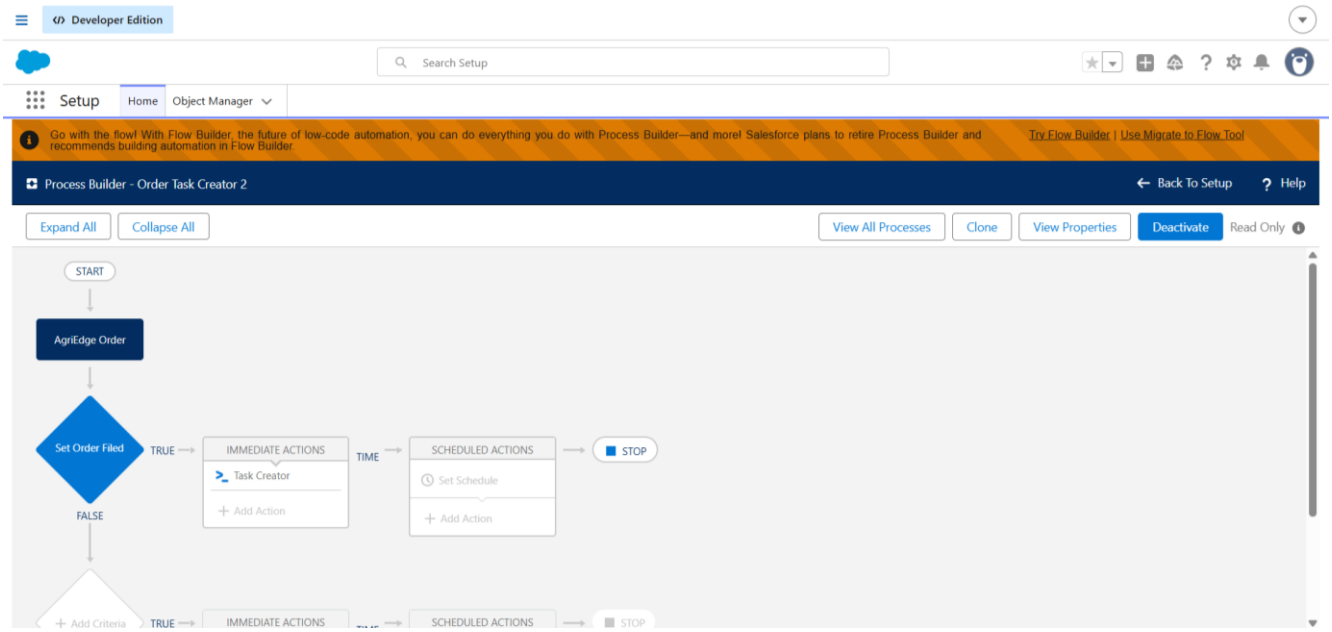
[SCREENSHOT 5 – AgriEdge Shipment Record]



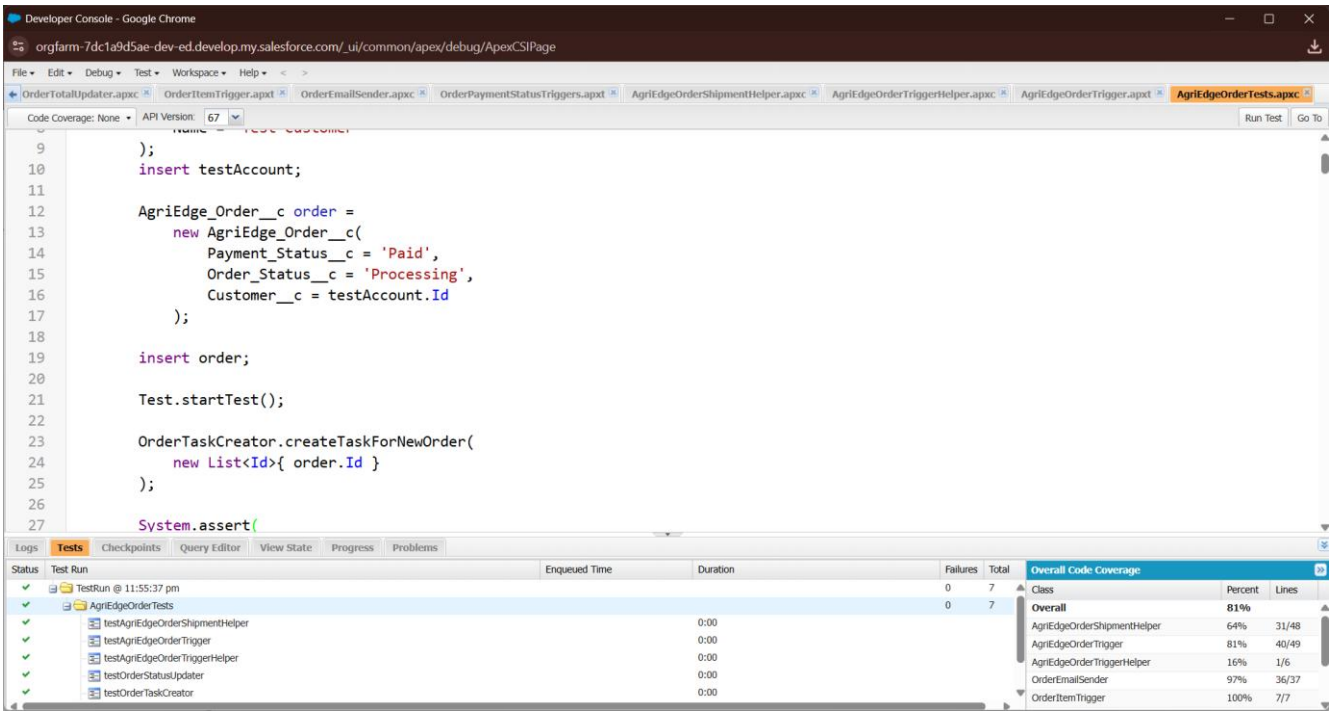
[SCREENSHOT 6 – AgriEdge Role Hierarchy]



[SCREENSHOT 7 –Order Task Creator Process Builder]



[SCREENSHOT 8 – AgriEdge Apex Test Results and Code Coverage]



.

9. Limitations of the Current Implementation

The current version of the AgriEdge Salesforce application does not include certain advanced features. The following functionalities have not been implemented and tested as part of the current project scope:

- Lightning Web Components (LWC): Custom web components for an enhanced user interface.
- Asynchronous Apex: Background processing using asynchronous Apex methods.
- Approval Process: Automated approval workflows for business transactions.
- Additional Automation: Workflow Rules or additional Flows beyond the implemented Process Builder.
- Data Migration: Data transfer using Data Import Wizard or Data Loader.
- Data Quality and History: Field History Tracking, Duplicate Rules and Matching Rules.
- Advanced Access Control: Permission Sets and Sharing Rules, unless configured separately.
- Reports and Dashboards: Analytical reports and dashboards, unless created and tested.

10. Proposed Future Improvements

We can add an LWC dashboard to monitor orders, inventory and shipments in real time. We can also add a chatbot for order-status and customer support queries. We can use AI for inventory suggestions and order prioritization. We can also add approval automation for high value orders and stronger data-quality rules. Finally, we can improve data migration and deployment for a production environment

11. Summary and Conclusion

In conclusion, the AgriEdge Salesforce application helps organize and manage customer details, orders, order items, inventory, and shipment information in a single platform. By implementing formula fields, access controls, Process Builder, and Apex automation, we have reduced manual effort and improved the efficiency of order processing. The Apex test class was executed successfully, with all seven tests passing and zero failures, achieving 76% overall code coverage. This project can be further improved in the future by introducing Lightning Web Components (LWC), analytics, AI-based features, and advanced automation.